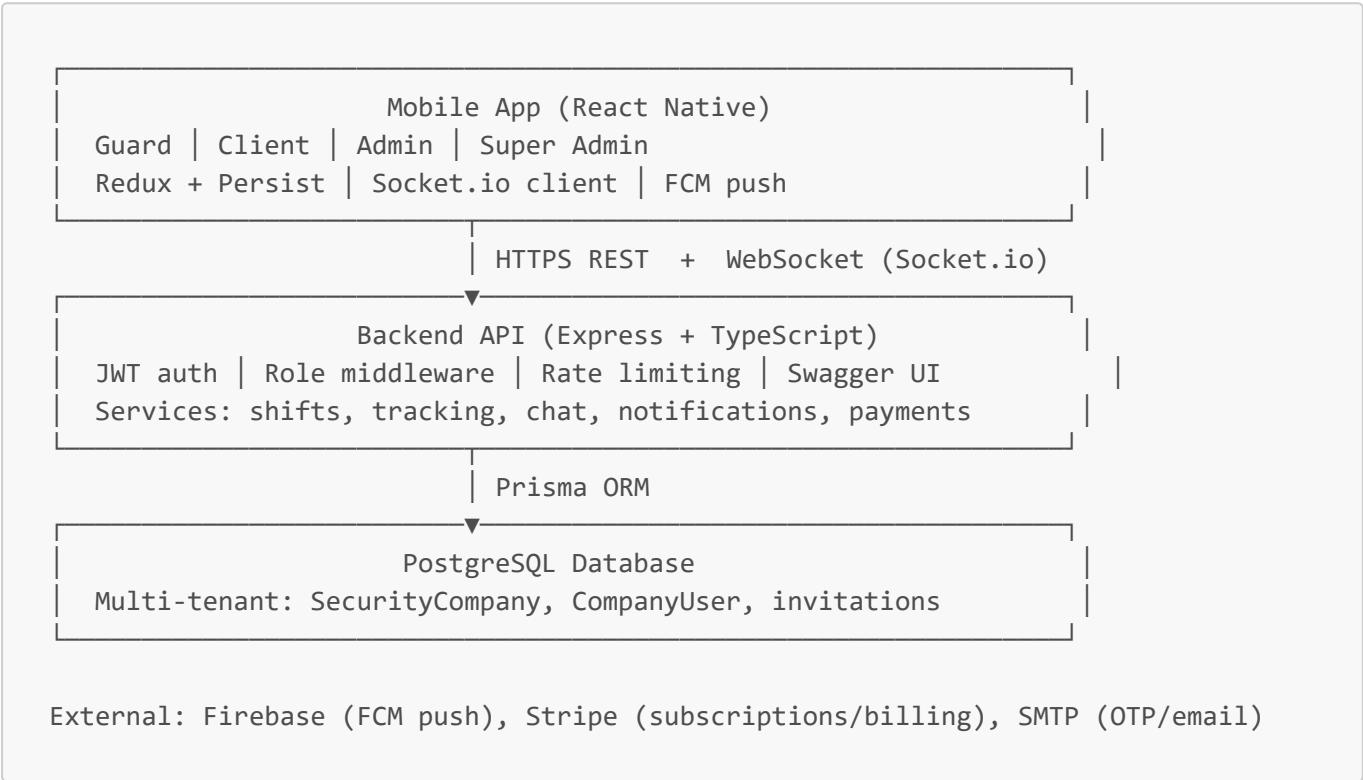


# System Architecture

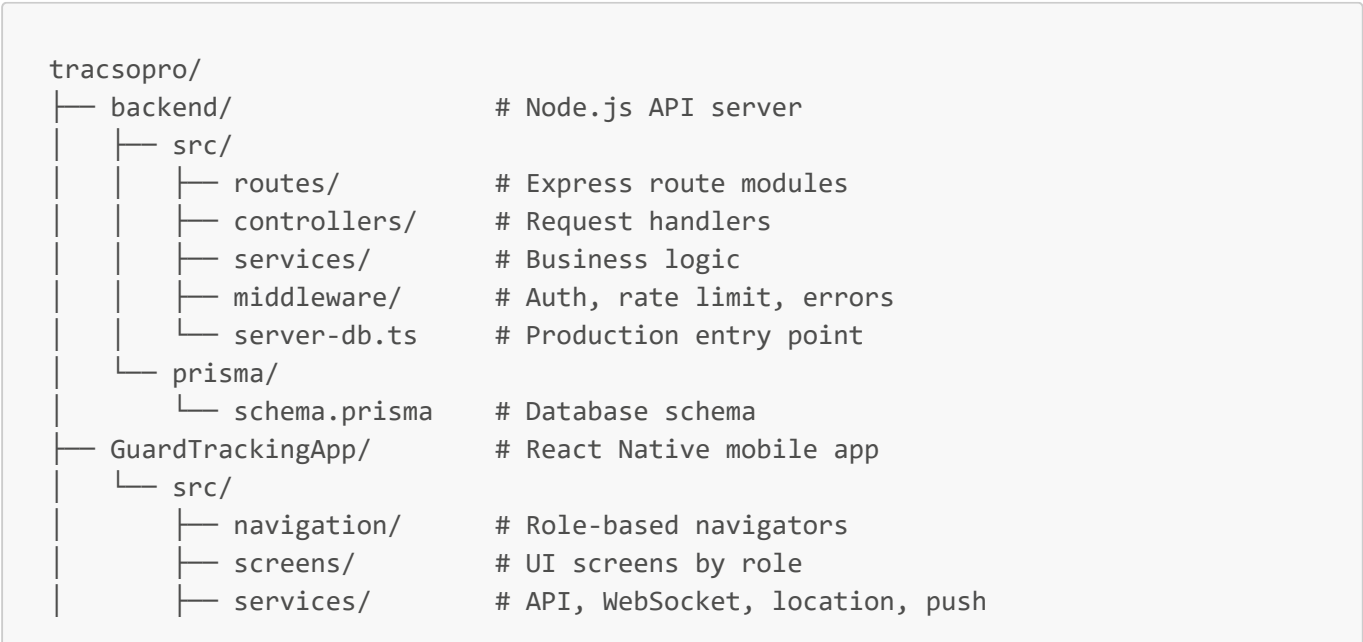
**Product:** TracSOPro — multi-tenant guard tracking platform  
**Last updated:** June 2026

## Overview

TracSOPro connects security companies, their guards, and client organizations through a React Native mobile app and a Node.js/PostgreSQL backend. Real-time location, shift lifecycle, incidents, chat, notifications, and billing are coordinated across four user roles.



## Repository layout



```

├── store/      # Redux slices
├── config/    # API/WebSocket URLs
└── docs/      # Architecture and reference docs

```

## User roles

| Role               | Description                      | Primary capabilities                                                     |
|--------------------|----------------------------------|--------------------------------------------------------------------------|
| <b>GUARD</b>       | Field security officer           | Shifts, check-in/out, location upload, incident reports, emergency alert |
| <b>CLIENT</b>      | Site owner / hiring organization | Sites, shift requests, guard visibility, reports, billing                |
| <b>ADMIN</b>       | Security company operator        | User/site/shift management, operations center, invitations               |
| <b>SUPER_ADMIN</b> | Platform operator                | Company onboarding, subscriptions, analytics, impersonation              |

Role is stored on `User.role` and enforced via `authenticate` + `authorize` middleware on the backend. The mobile app routes authenticated users to role-specific navigators in `MainNavigator.tsx`.

## Multi-tenancy

Each security company is a `SecurityCompany` record. Users link to a company through `CompanyUser`. Guards, clients, and sites are associated via `CompanyGuard`, `CompanyClient`, and `CompanySite`. Invitations (`Invitation`) allow controlled onboarding of guards and clients into a tenant.

Subscription limits (`maxGuards`, `maxClients`, `maxSites`) and Stripe billing are scoped per company.

## Core domain flows

### Authentication

1. User registers or accepts an invitation → OTP email verification.
2. Login returns JWT access token + refresh token.
3. Mobile stores tokens (Keychain/AsyncStorage via Redux persist).
4. API requests send `Authorization: Bearer <token>`.
5. WebSocket connects and emits `authenticate` with the same JWT.

### Shift lifecycle

```

SCHEDULED → IN_PROGRESS (check-in) → ON_BREAK (optional) → COMPLETED (check-out)
    ↘ CANCELLED / NO_SHOW / EARLY_END

```

- **Client** or **Admin** creates shifts tied to a site.
- **Admin** can assign guards to unassigned shifts.

- **Guard** checks in/out with GPS validation against site geofence (**radiusMeters**).
- Breaks recorded as **ShiftBreak** records.

## Location tracking

1. Guard app uploads GPS via REST (**POST /api/tracking/location**) and/or WebSocket (**location\_update**).
2. Backend persists **TrackingRecord** and broadcasts to admin/client viewers.
3. Periodic **live\_locations\_update** broadcast for operations dashboards.
4. Geofence enter/exit events stored as **GeofenceEvent**.

## Incidents and reports

- **Shift incidents** — tied to an active shift (**ShiftIncident**).
- **Incident reports** — standalone guard submissions with media (**IncidentReport** + **IncidentReportMedia**).
- **Legacy incidents** — location-based **Incident** model with evidence attachments.
- Admins and clients can respond to and review reports.

## Emergency alerts

Guards trigger **POST /api/emergency/alert**. Admins and clients receive push + WebSocket **emergency\_alert**. Acknowledge and resolve workflows are role-gated.

## Chat and support

- Direct and group conversations (**Conversation**, **ConversationParticipant**, **Message**).
- Support tickets (**SupportTicket**) with company vs platform audience.
- Real-time delivery via WebSocket room join (**join\_room** / **chat:<id>**).

## Notifications

- In-app notifications stored in **Notification**.
- FCM push via Firebase Admin (**DeviceToken** registration from mobile).
- Retry queue (**PushNotificationRetry**) for failed deliveries.

## Billing

- Stripe subscriptions per **SecurityCompany**.
- Checkout, billing portal, invoices, and webhooks at **/api/payments/\***.
- Super Admin manages company subscriptions and payment status.

## Backend architecture

| Layer              | Responsibility                                                                  |
|--------------------|---------------------------------------------------------------------------------|
| <b>Routes</b>      | HTTP method + path mapping, Swagger annotations                                 |
| <b>Middleware</b>  | <b>authenticate</b> , <b>authorize(roles)</b> , rate limiting, error handling   |
| <b>Controllers</b> | Parse request, call services, format { <b>success</b> , <b>data</b> } responses |

| Layer    | Responsibility                                        |
|----------|-------------------------------------------------------|
| Services | Business logic, Prisma queries, external integrations |
| Prisma   | Type-safe database access                             |

**Entry point:** `backend/src/server-db.ts` — connects DB, initializes Firebase, WebSocket, background jobs (push retry, token cleanup, live location broadcast).

**Legacy:** `backend/src/server.ts` in-memory server exists for compatibility; do not use in production.

**Interactive API docs:** Swagger UI at `/api-docs` when backend is running.

## Mobile architecture

| Layer       | Responsibility                                                                                                                       |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Navigation  | <code>AppNavigator</code> → <code>AuthNavigator</code> / <code>MainNavigator</code> → role stacks                                    |
| Redux store | Auth, shifts, location, chat, notifications, admin, client slices                                                                    |
| Services    | <code>api.ts</code> (Axios), <code>WebSocketService</code> , <code>locationTrackingService</code> , <code>notificationService</code> |
| Hooks       | <code>useGuardLocationTracking</code> , <code>useSubscriptionLimits</code> , <code>useDataSync</code>                                |

**Entry point:** `GuardTrackingApp/App.tsx` — Redux Provider, PersistGate, ThemeProvider, AppNavigator.

**Config:** `src/config/apiConfig.ts` — API and WebSocket base URLs (must use HTTPS in production).

## Security model

- Passwords hashed with bcrypt.
- JWT access tokens (short-lived) + refresh tokens (stored in DB, revocable).
- WebSocket requires valid JWT on `authenticate` event; user/role mismatch rejected.
- Helmet, CORS (restricted in production), API rate limiting.
- Tenant isolation enforced in services (see `backend/tests/tenantIsolation.test.ts`).

## Realtime events (WebSocket)

| Client → Server                                  | Server → Client                                                    |
|--------------------------------------------------|--------------------------------------------------------------------|
| <code>authenticate</code>                        | <code>authenticated</code> , <code>authentication_error</code>     |
| <code>location_update</code>                     | <code>live_locations_update</code> , <code>location_updated</code> |
| <code>geofence_event</code>                      | <code>geofence_event</code>                                        |
| <code>emergency_alert</code>                     | <code>emergency_alert</code> (to admins)                           |
| <code>shift_status_update</code>                 | <code>shift_status_update</code>                                   |
| <code>request_live_locations</code>              | <code>live_locations_data</code>                                   |
| <code>join_room</code> / <code>leave_room</code> | <code>room_joined</code> , <code>room_left</code> , chat events    |

**Client → Server**

**Server → Client**

---

— notification

Related documents

- [API\\_REFERENCE.md](#)
- [DATABASE\\_SCHEMA.md](#)
- [MOBILE\\_APP\\_ARCHITECTURE.md](#)
- [DEPLOYMENT\\_GUIDE.md](#)